

AFRILANG Stdlib

std/c/nlptfidf

Français. Bibliothèque AFRILANG 'std/c/nlptfidf' (niveau complex, catégorie complex). Elle expose 6 fonction(s) : tf, idf, tfidf, docVector, cosineSim, rankTerms.

```
'import "std/c/nlptfidf"' · 'use nlptfidf'
```

```
- 'tf(doc text, term text) → number' - 'idf(term text, corpus list text) → number' - 'tfidf(doc text, term text, corpus list text) → number' - 'docVector(doc text, terms list text, corpus list text) → list number' - 'cosineSim(a list number, b list number) → number' - 'rankTerms(doc text, corpus list text) → list text'
```

English. AFRILANG library 'std/c/nlptfidf' (tier complex, category complex). Exports 6 function(s): tf, idf, tfidf, docVector, cosineSim, rankTerms.

```
'import "std/c/nlptfidf"' · 'use nlptfidf'
```

```
- 'tf(doc text, term text) → number' - 'idf(term text, corpus list text) → number' - 'tfidf(doc text, term text, corpus list text) → number' - 'docVector(doc text, terms list text, corpus list text) → list number' - 'cosineSim(a list number, b list number) → number' - 'rankTerms(doc text, corpus list text) → list text'
```

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	6	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/nlptfidf' (niveau complexe, catégorie complexe). Elle expose 6 fonction(s) : tf, idf, tfidf, docVector, cosineSim, rankTerms.

```
'import "std/c/nlptfidf"' · 'use nlptfidf'
```

```
- `tf(doc text, term text) → number`
- `idf(term text, corpus list text) → number`
- `tfidf(doc text, term text, corpus list text) → number`
- `docVector(doc text, terms list text, corpus list text) → list number`
- `cosineSim(a list number, b list number) → number`
- `rankTerms(doc text, corpus list text) → list text`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/nlptfidf"
use nlptfidf
```

Niveau : complexe · Catégorie : Modules complexe (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- tf(doc text, term text) → number•
idf(term text, corpus list text) → number•
tfidf(doc text, term text, corpus list text) → number•
docVector(doc text, terms list text, corpus list text) → list•
cosineSim(a list number, b list number) → number•
rankTerms(doc text, corpus list text) → list

4. Exemples d'utilisation

```
import "std/c/nlptfidf"
use nlptfidf
```

```
create result = tf(/* args */)
say result
```

```
create result = idf(/* args */)
say result
```

```
create result = tfidf(/* args */)
say result
```

```
create result = docVector(/* args */)
say result
```

```
create result = cosineSim(/* args */)
say result
```

```
create result = rankTerms(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/nlptfidf"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module nlptfidf

```
function tf(doc text, term text) returns number  
end
```

```
function idf(term text, corpus list text) returns number  
end
```

```
function tfidf(doc text, term text, corpus list text) returns number  
end
```

```
function docVector(doc text, terms list text, corpus list text) returns list number  
end
```

```
function cosineSim(a list number, b list number) returns number  
end
```

```
function rankTerms(doc text, corpus list text) returns list text  
end  
end
```

English

1. Overview

AFRILANG library 'std/c/nlptfidf' (tier complex, category complex). Exports 6 function(s): tf, idf, tfidf, docVector, cosineSim, rankTerms.

```
'import "std/c/nlptfidf"' · 'use nlptfidf'
```

```
- `tf(doc text, term text) → number`
- `idf(term text, corpus list text) → number`
- `tfidf(doc text, term text, corpus list text) → number`
- `docVector(doc text, terms list text, corpus list text) → list number`
- `cosineSim(a list number, b list number) → number`
- `rankTerms(doc text, corpus list text) → list text`
```

2. Installation & import

Add the module to your program:

```
import "std/c/nlptfidf"
use nlptfidf
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- tf(doc text, term text) → number•
idf(term text, corpus list text) → number•
tfidf(doc text, term text, corpus list text) → number•
docVector(doc text, terms list text, corpus list text) → list•
cosineSim(a list number, b list number) → number•
rankTerms(doc text, corpus list text) → list

4. Usage examples

```
import "std/c/nlptfidf"
use nlptfidf
```

```
create result = tf(/* args */)
say result
```

```
create result = idf(/* args */)
say result
```

```
create result = tfidf(/* args */)
say result
```

```
create result = docVector(/* args */)
say result
```

```
create result = cosineSim(/* args */)
say result
```

```
create result = rankTerms(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/nlptfidf"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module nlptfidf

```
function tf(doc text, term text) returns number
end
```

```
function idf(term text, corpus list text) returns number
end
```

```
function tfidf(doc text, term text, corpus list text) returns number
end
```

```
function docVector(doc text, terms list text, corpus list text) returns list number
end
```

```
function cosineSim(a list number, b list number) returns number
end
```

```
function rankTerms(doc text, corpus list text) returns list text
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/nlptfidf"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/nlptfidf/>

Source (extrait)

```
module nlptfidf

function tf(doc text, term text) returns number
end

function idf(term text, corpus list text) returns number
end

function tfidf(doc text, term text, corpus list text) returns number
end

function docVector(doc text, terms list text, corpus list text) returns list number
end

function cosineSim(a list number, b list number) returns number
```

```
end  
function rankTerms(doc text, corpus list text) returns list text  
end  
end
```